

Everyday Dental Surgery — Comprehensive Technical Overview

Release summary

September 2026 • Synthetic portfolio release • Client and developer edition

Everyday Dental demonstrates a fictional dental-practice website, an interactive sample patient journey and an evidence-based engineering plan. It gives reviewers a working interface and inspectable implementation without asking them to submit real health information. The public deployment is available at [Open the dental demonstration](#).

The release includes bilingual presentation, responsive layouts, sample privacy controls, explicit access-denial examples, first-party media, restrictive browser headers and an isolated static origin. It also preserves a historical Supabase clinical prototype for technical review. All 11 historical Edge Function endpoints are now disabled by a deployed guard following explicit owner approval. Existing database records were preserved; no migrations or database mutation commands were executed.

This is not a working clinical patient portal, HIPAA/GDPR certification, full WCAG conformance statement or proven million-user platform. The value for a client is the separation of implemented behavior, tested evidence, known defects and future acceptance gates. Independent review is intentionally reserved for a separate session.

A practical demonstration for reviewers

Start on the home page and review the fictional clinic presentation. Open the sample patient journey from the demonstration notice. The page uses a fixed synthetic patient and makes its nonclinical status explicit. The role selector illustrates policy decisions; visitors are not authenticating as real staff.

With the patient role, review the synthetic record, change optional sharing, cancel the sample appointment and download a FHIR-shaped Patient bundle. Optional sharing begins off. Try erasure while the legal hold is active: the operation is deferred. Switch to the sample admin role to release the hold, return to the patient role and erase the sample. Export is then denied until reset. Switch to reception to observe that clinical access and patient consent changes are denied.

These interactions are browser-memory transitions, not authoritative clinical decisions. The activity list is a temporary illustration, not an immutable audit trail. Closing or resetting the page clears the sample state. The downloaded file contains the fixed sample only. Real identity, care assignment, tenant boundaries, lawful retention decisions and durable audit records belong to the future server implementation.

Visit the privacy and engineering page to examine the stated limits and accessibility behavior. The contact, registration, appointment and login layouts remain read-only previews. No clinical client, real payment, email, SMS or live patient intake is enabled by the public application.

Product surfaces and technical structure

The presentation retains home, about, services and service-detail pages; pricing; articles; FAQ; contact; gallery; community; conferences; appointment previews; account-form previews; and

privacy/terms routes. Clinic profiles, testimonials, offers, treatment figures and pricing are fictional demonstration content. They must be replaced with verified, jurisdiction-appropriate business content before a real clinic uses the design.

React handles page composition and state. React Router provides navigation and restricted-portal redirects. Vite produces versioned assets. Tailwind and existing motion libraries preserve the design; reduced-motion preferences are respected. The obsolete animation dependency that produced an eval warning was removed. Forty-seven remote stock-image variants are now served from the same site, with source and license information retained.

The new configuration boundary validates the public HTTPS origin and accepts synthetic mode only. Sample records, workflow messages, disclosures and image mappings have maintained data files. Deployment settings own the pinned server image, resource bounds and cache settings. Tooling settings own local test limits and capacity assumptions. Legacy clinical configuration and older embedded presentation content still need consolidation before clinical reuse.

The public request path is browser → CDN → HTTPS gateway → loopback static container. No patient query is needed to render a page or execute the synthetic journey. The origin runs as a non-root user with a read-only filesystem, dropped capabilities and explicit resource limits. Only the static build and generated server configuration are deployed; credentials, SQL, source and recovery files are excluded from the web root.

Historical clinical architecture and engineering debt

The preserved prototype contains 11 migrations and 11 Edge Functions covering form submission, patient data, cancellation, administrative queries and role management, request resolution, FHIR access/export, notifications, breach checks and payments. Earlier modules also sketch encrypted fields, database authorization, consent, retention and audit concerns. Source presence is not evidence that those concerns are correctly implemented end to end.

The prior implementation had material contract and security gaps: plaintext writers did not consistently match the encrypted schema, patient-controlled query selection and FHIR ownership needed repair, privileged notification/breach operations required stronger caller authority, payment verification and idempotency were incomplete, and rate limiting could fail open. The release guard contains these functions; it does not repair those defects or establish that the database REST/Auth/Storage configuration is safe for real records.

Clinical activation requires its own isolated test database, verified schema upgrade path, complete server-side authorization, operational contracts, region decisions and independent security review. Removing a guard is not an acceptable shortcut. The clinical-release section later in this report lists the repair and verification obligations in detail.

Engineering approach and handoff

The implementation uses small, testable policy functions and explicit boundaries rather than simulated production credentials. Real browser checks complement unit tests. Deployment work begins with a live snapshot and local comparison, then a versioned local build, runtime checks and deployed-file hashes. Recovery archives and project checkpoints allow another developer to resume after workstation power loss.

The remaining sections provide the legal research, security model, staged capacity and availability design, operational runbook, clinical release gates and exact verification limits. They should be

read together: a law citation does not prove compliance, a passing test does not prove an SLA, and a planning calculation does not prove production capacity.

US and EU privacy, health-data and accessibility readiness

Executive assessment

Everyday Dental is a fictional portfolio demonstration. Its useful professional claim is that it demonstrates selected privacy and security decisions, documents their limits, and provides a reviewable route to a production system. It is not a certified clinical platform. The sample patient journey uses fixed synthetic records in browser memory. Public intake is disabled, and the shipped frontend cannot connect to the historical clinical backend through environment variables.

This distinction is important commercially. A client can examine the design, tests and release evidence without trusting invented compliance badges. The architecture supports a future separated clinical service, but deploying that service is a new engineering and governance milestone. It is not a configuration toggle. Findings below distinguish legal requirements, applicability questions and project recommendations. Research was checked against primary sources in September 2026; jurisdiction and product changes require reassessment.

Scope and applicability

The baseline covers US federal health/privacy law, California and Washington examples, EU GDPR and ePrivacy, accessibility, and future European health-data interoperability. It does not purport to cover every US state or every EU member state. A real launch needs a named operating entity, patient locations, service locations, data flows, vendors and purposes before counsel can complete the applicability analysis.

- Deployment: Public portfolio • Recommended boundary: Fictional content, no real patient intake, no payment or messaging • Additional review before launch: Truthful claims, accessibility, hosting metadata and asset rights
- Deployment: Clinic brochure/contact site • Recommended boundary: Minimum administrative contact data • Additional review before launch: Advertising, privacy notice, retention, tracking, patient expectations
- Deployment: Patient portal • Recommended boundary: Verified identity and server-side access controls • Additional review before launch: Health confidentiality, contracts, consent/legal basis, rights and incident handling
- Deployment: Multi-clinic SaaS • Recommended boundary: Tenant-scoped clinical service • Additional review before launch: Processor/business-associate obligations, regional hosting, auditability, vendor chain
- Deployment: Clinical AI or diagnostic software • Recommended boundary: Separate product classification • Additional review before launch: Medical-device and AI-specific assessment; no inference from this demo

Synthetic records do not eliminate all privacy considerations: hosting providers still handle network traffic. The release avoids application analytics and automatic third-party media loads. An operator must still understand provider security logs, contractual roles, retention and cross-border processing. “No patient database” is narrower and more defensible than “we process no personal data.”

United States

HIPAA applicability and contracts

HIPAA applies to covered entities and business associates, not automatically to every health-themed website. A healthcare provider conducting covered electronic transactions may be a covered entity. A vendor handling protected health information on its behalf may be a business associate. Cloud storage can create business-associate responsibilities even when a provider does not hold the decryption key. Appropriate BAAs and a shared responsibility allocation are essential before a relevant cloud service handles ePHI.[1]

Project decision: do not accept real patient records on the demo. Do not describe HTTPS, encryption libraries or a policy folder as proof of HIPAA compliance. A real engagement should maintain a vendor matrix recording service, purpose, data classes, BAA status, subcontractors, access roles and termination/export process.

Supabase's documented HIPAA configuration requires a signed BAA and its HIPAA add-on for PHI, with designated high-compliance projects. Its shared-responsibility documentation identifies the required plan level. The old repository instruction that an ordinary Pro subscription alone enables HIPAA use was therefore unsuitable. No paid plan or add-on is activated by this work.[2]

Security Rule and evidence

HHS describes administrative, physical and technical safeguards, risk analysis, access management, contingency planning and evaluation. The Security Rule's documentation retention requirement is distinct from a clinic's medical-record retention schedule. A universal "delete all health records after six years" rule would be an unsafe interpretation.[3]

Recommended clinical evidence: maintain an asset inventory and risk register; require staff MFA; verify role and patient/tenant scope on every request; rotate and recover encryption keys; review audit access; rehearse restoration; train workforce members; review vendors; and record incident decisions. Access-control code needs negative tests, including another patient, another tenant, expired sessions, revoked roles and failed audit persistence. Browser role selectors and inactivity timers are not server enforcement.

HHS still describes the Security Rule cybersecurity update as a proposed rule and states that the current rule remains in effect during rulemaking. Proposed MFA/encryption requirements should not be represented as enacted solely because they are desirable design choices. Track the rulemaking at release time.[4]

Health-data breach response

HIPAA breach notification generally requires individual notification without unreasonable delay and no later than 60 days after discovery; reporting obligations differ with breach size and circumstances. This is an outer deadline, not a recommendation to wait. Document assessment, discovery, containment, affected data, recipient decisions and the basis for notification.[5]

The FTC's Health Breach Notification Rule can cover qualifying personal-health-record vendors and related entities outside HIPAA. The 2024 amendments clarify coverage and unauthorized disclosures. A health app should not assume that exclusion from HIPAA means no health-breach obligations. Applicable organizations need an incident path aligned with the rule, alongside any state duties.[6]

Recommended implementation: retain a restricted incident register with discovery time and jurisdiction-specific deadlines, identity of the incident owner, evidence links and approval history. Compute deadlines by jurisdiction; do not hardcode a single “60-day compliance timer” for all incidents. Protect evidence from routine purges and keep notification drafts separate from ordinary product email.

California

The CCPA applies to covered businesses under its applicability tests; rights and exceptions are contextual. Relevant concepts include disclosure, access, deletion, correction, sale/sharing opt-out, sensitive-information limits and non-discrimination. Medical-information exemptions do not justify assuming every dataset held by a clinic is exempt. Advertising and website activity need their own analysis.[7]

California’s Confidentiality of Medical Information Act separately restricts disclosure of medical information and defines authorization conditions and exceptions. CCPA checklists do not replace medical-confidentiality analysis.[8]

Project decision: no application sale/sharing for advertising, analytics pixels or replay. A Global Privacy Control signal cannot trigger tracking because no application tracking path exists. A real clinic must supply verified privacy contacts, request verification and response procedures, authorized-agent handling and a record-category exemption analysis. Do not publish invented contact details as a working rights-request channel.

Washington

Washington’s My Health My Data Act addresses consumer health data beyond traditional HIPAA coverage. Its text includes consumer-health notices, consent rules, rights, security practices, processor duties, separate authorization for sale and healthcare geofencing restrictions. Scope includes certain residents and health data collected in Washington; exemptions need a data-specific assessment. A broad terms checkbox is not a substitute for the required consent.[9]

Project decision: no geolocation/geofencing, no advertising sale, no embedded map and optional sharing off in the sample. A real product should inventory inferred health data and acquisition sources as well as direct clinical fields. Evaluate the exact consent, deletion, backup and appeal workflow under the statute; do not assume a generic GDPR screen is enough.

European Union

GDPR legal bases and responsibility

GDPR applicability depends on establishment and relevant offering/monitoring activities, not merely whether a site is reachable in Europe. Health data are a special category. A real clinical service needs an Article 6 basis and a valid Article 9 condition, with national healthcare/secrecy requirements considered. Consent is not the universal basis for all treatment processing. Privacy by design, minimization, transparency, retention, processor contracts and risk-appropriate safeguards must be reflected in the actual data lifecycle.[10]

Recommended design: separate treatment records from marketing preferences, retain the purpose and legal basis of each processing activity, and require explicit approval when adding a new use. Record why a field is necessary. An intake form should not demand clinical history when a

scheduling request needs only an appointment category and a secure callback channel. Avoid sending clinical text in URLs, analytics labels, emails or support screenshots.

The sample's optional sharing toggle intentionally does not control the legal basis for all treatment. Its legal hold illustrates why an erasure request may need review. These are educational workflow examples; they do not decide a real person's legal entitlement.

Security, incidents and international transfers

The EDPB's small-business guidance connects security to organizational and technical measures, including access control, backups and incident response. Under GDPR, an applicable controller generally notifies the authority within 72 hours after becoming aware of a qualifying personal-data breach, unless the risk exception applies; high-risk individual communication is a separate assessment. Keep the reasoning and evidence, including decisions not to notify.[11]

Hosting in an EU region alone does not settle international transfers. Review support access, subprocessors and onward transfers, then identify a valid transfer mechanism where required. The EDPB describes mechanisms and safeguards for transfers outside the EEA; assess the actual vendor arrangement and supplementary measures rather than treating geography as certification.[12]

Recommended clinical implementation: region-specific storage boundaries, least-privilege support access, encryption with documented key ownership, restricted exports, auditable access requests and a vendor exit plan. Test restoration into the authorized region. Do not put genuine patient records into a public object cache or an unapproved observability provider.

ePrivacy and browser storage

CNIL guidance explains prior consent for non-essential trackers, with exceptions for necessary functions, and requires a meaningful ability to refuse and withdraw. Broad acceptance of terms does not establish valid tracker consent. Language preferences may be an expected interface function, but that is not permission for analytics or fingerprinting.[13]

Implemented design: first-party stock images, no automatic external map, no advertising/analytics scripts, no patient information in persistent browser storage, and a language-preference clearing control. The sample resets when the tab reloads. Because there are no optional tracking technologies, adding a decorative "accept all cookies" banner would misdescribe this product. If tracking is added later, inventory it first, gate it before consent where required, and test refusal and withdrawal equally carefully.

Accessibility and member-state variation

US DOJ guidance identifies accessibility obligations for covered public accommodations and discusses barriers such as missing alternatives, poor contrast and inaccessible forms. A private clinic's duties should not be confused with the separate Title II rules for state/local government services.[14]

The European Accessibility Act applies to specified products and services, including relevant e-commerce services; it is not an automatic blanket rule for every brochure website. Its scope, microenterprise service exemptions and national implementation need assessment. National disability and healthcare duties may still apply independently.[15]

WCAG 2.2 provides a technical standard for evaluating accessibility. Keyboard operability, visible focus, labels, status announcements, text alternatives, contrast, reflow and motion preferences are practical test targets. Automated checks only cover part of accessibility; manual keyboard and assistive-technology evaluation remain necessary.[16]

Implemented/recommended boundary: the new review pages include semantic headings, keyboard controls, visible focus, a skip link and announced workflow results. Reduced motion is supported. Legacy cinematic pages need broader manual evaluation, including zoom, screen readers, vestibular comfort and accessible error recovery. Do not display a conformance badge based only on an automated scan.

Before onboarding a clinic in a named member state, confirm professional registration/advertising requirements, minor-consent rules, record-retention periods, secrecy obligations, accessibility rules, required site legal notices, supervisory authority and relevant national health-hosting requirements. These remain a launch checklist because no target country or actual clinic was specified.

EHDS and future product expansion

The European Commission describes phased application of the European Health Data Space, including major primary-use milestones in 2029 and 2031. A FHIR-shaped JSON export does not prove EHDS interoperability, conformity assessment, national integration or patient identity matching.[17]

The demonstration makes no clinical AI decision, diagnosis, prescription or medical-device claim. Adding AI or clinical decision support requires a separate intended-purpose and regulatory-classification review. The project should not invent an AI Act or medical-device certification for functionality it does not implement.

Implementation matrix

- Concern: Patient data collection • This public release: Disabled; fixed synthetic workflow • Production evidence still required: Final schema, lawful purpose, data minimization and negative authorization tests
- Concern: Identity and access • This public release: Sample role-policy model • Production evidence still required: Verified identity, MFA, tenant/patient scope, session revocation, care-assignment policy
- Concern: Consent • This public release: Optional sharing off; reversible sample toggle • Production evidence still required: Versioned notices, proof, purpose separation and downstream withdrawal propagation
- Concern: Erasure/retention • This public release: Sample hold and erasure transition • Production evidence still required: Legal schedule, restricted review, backup strategy and verified deletion/exception response
- Concern: Export • This public release: Synthetic Patient bundle • Production evidence still required: Partner implementation guide, authorized scope, terminology, validation and secure delivery
- Concern: Audit • This public release: Temporary sample outcomes • Production evidence still required: Restricted durable server events, tamper evidence, monitoring and retention
- Concern: Tracking • This public release: No application analytics or advertising • Production evidence still required: Provider inventory and approval before adding new processors

- Concern: Encryption • This public release: HTTPS and no clinical database in demo • Production evidence still required: Key management, encrypted storage/backups, recovery and rotation tests
- Concern: Incidents • This public release: Documented response design • Production evidence still required: Named responders, tested deadlines, evidence handling and exercises
- Concern: Availability • This public release: Stated target and repeatable probes • Production evidence still required: Sustained measurements and funded redundancy for stronger guarantees

Operating model and client acceptance

The operator should own legal applicability, approved vendors, retention decisions and notices. Engineering owns the implemented controls and reproducible evidence. A privacy/security lead owns risk review and incident coordination. These responsibilities need named people in a real engagement; a repository cannot appoint an absent compliance officer.

A client acceptance review should walk through one normal and one denied action per role, one withdrawal, one export, one hold/deletion request and one restore. Review actual network requests and logs. Verify that a changed role takes effect immediately and that failure of a dependency does not open access. Budget and contracts must be approved before sending real data to any new service.

Useful commercial wording is: “Synthetic dental-workflow demonstration with privacy-conscious delivery, tested sample access policies, documented production gates and a staged scaling design.” Avoid “HIPAA certified,” “GDPR compliant worldwide,” “SOC 2 audited,” “one million concurrent users supported,” or “guaranteed uptime” without the corresponding independent and operational evidence.

Sources

1. HHS. Guidance on HIPAA and Cloud Computing.
2. Supabase. HIPAA Projects and Shared Responsibility Model.
3. HHS. Summary of the HIPAA Security Rule.
4. HHS. HIPAA Security Rule NPRM. Proposed rule; current rule remains applicable during rulemaking.
5. HHS. Breach Notification Rule.
6. FTC. Complying with the Health Breach Notification Rule. Includes amendments effective in 2024.
7. California DOJ. California Consumer Privacy Act.
8. California Legislature. Civil Code, Confidentiality of Medical Information, Chapter 1.
9. Washington Legislature. RCW 19.373, My Health My Data Act.
10. European Union. Regulation (EU) 2016/679, especially Articles 3, 5, 6, 9, 17, 25, 28, 32–35 and 44 onward.
11. EDPB. Secure personal data.
12. EDPB. International data transfers.
13. CNIL. Cookies et traceurs : que dit la loi ?.
14. US DOJ. Guidance on Web Accessibility and the ADA.
15. European Union. Directive (EU) 2019/882, European Accessibility Act.
16. W3C. Web Content Accessibility Guidelines 2.2.

Threat model and release boundaries

Assets and trust boundaries

The public release contains fictional clinic content, stock media, a synthetic Patient fixture, deployment configuration and source code. It holds no clinical account session, payment key or patient database connection. A visitor controls their browser, URL and role selector. Browser state must therefore never be trusted as production identity or permission.

Trust boundaries are: visitor → CDN; CDN/direct-origin visitor → TLS gateway; gateway → loopback static container; static files → browser; operator workstation → pinned SSH deployment; and maintainer → Git/CI. The preserved Supabase prototype is a separate, unapproved clinical surface. Its runtime controls cannot be inferred from the public static demo.

Threat and control matrix

- Threat: Accidental patient intake • Implemented control: Disabled form previews; no backend client; fixed fixture-only sample • Verification / remaining boundary: Browser checks find no enabled personal-data inputs; guard regression tests
- Threat: Unauthorized clinical backend activation • Implemented control: Unconditional shared release guard on every historical Edge endpoint • Verification / remaining boundary: Local boundary tests; remote deployment must be recorded separately
- Threat: XSS / injected dependencies • Implemented control: No inline scripts/eval permission; React escaping; dependency updates; strict CSP • Verification / remaining boundary: Header/browser checks and npm audit; style-src permits inline styles for current UI
- Threat: Tracking and third-party disclosures • Implemented control: First-party image variants; no automatic fonts/maps/analytics; no-referrer • Verification / remaining boundary: Browser request inventory; network metadata still reaches hosting/CDN
- Threat: Client patient-data persistence • Implemented control: Retired storage mock; sample memory only; old app cache retirement • Verification / remaining boundary: Source and browser tests; exported sample file deliberately downloads to user device
- Threat: Clickjacking • Implemented control: frame-ancestors none and X-Frame-Options DENY • Verification / remaining boundary: HTTP headers
- Threat: Secret/source exposure • Implemented control: Static artifact allowlist, dotfile/source/config exclusions, no source maps • Verification / remaining boundary: Negative HTTP path tests and staged/history secret scan
- Threat: Origin/container compromise • Implemented control: Non-root, read-only filesystem, dropped capabilities, no-new-privileges, bounded resources, loopback-only publishing • Verification / remaining boundary: Compose validation and actual runtime inspection
- Threat: Resource exhaustion • Implemented control: Static-only request surface, GET/HEAD, request/body time limits, bounded resource allocation • Verification / remaining boundary: Does not provide a contractual DDoS guarantee; no large shared-origin load test
- Threat: Supply-chain regression • Implemented control: Lockfile, dependency audit, gitleaks, typed new boundary, policy tests and lint • Verification / remaining boundary: Review registry updates and independent review before production

- Threat: Unsafe deployment/rollback • Implemented control: Pre-edit drift snapshot, versioned release, exact hashes, off-server archive • Verification / remaining boundary: Verify restore and record release before advertising completion
- Threat: Misleading compliance claims • Implemented control: Research-based limits, clinical blocker register, measured vs target separation • Verification / remaining boundary: No certifications, million-user claim or observed monthly uptime without evidence

Security assumptions and residual risks

TLS gateway administration is shared with other applications. The VPS remains a single failure domain, and the existing infrastructure permits origin web access. Cloudflare does not replace origin authorization. This demo has no sensitive API to authenticate, but a future clinical service must enforce authorization even when the CDN is bypassed.

The sample audit list is temporary and editable by a browser owner. It is an illustration, not a protected audit trail. The sample staff policy is intentionally simplified and does not model care assignment, tenant membership or emergency access. Those are production release blockers.

Original clinical modules and migrations remain readable for design review, with known defects documented. Their presence is not shipped clinical functionality. A source scanner score from compliance/ is a historical pattern check; it does not measure legal compliance, attack resistance or runtime correctness.

Incident and change procedure

For a suspected compromise: preserve evidence privately; disable the affected route/service using a reviewed local change; identify affected assets and versions; compare deployment hashes; rotate exposed credentials through approved stores; assess legal notification obligations; restore a verified release; and add a defect-log entry identifying the missed gate. Do not publish secrets or patient content in issues or commit messages.

Before changes: update the project checkpoint, research the approach, inventory configurable values and write acceptance tests. After changes: run relevant gates, record before/after evidence, update the dossier first, and derive public statements from verified results.

Reference framework: OWASP Application Security Verification Standard. The project borrows review concepts; it does not claim full ASVS verification.

Capacity and availability design

Current release

The public release serves static HTML, JavaScript, CSS and images. The sample patient journey executes in browser memory and causes no clinical database requests. This keeps a demo independent of a paused database and avoids maintaining patient data merely to demonstrate design. It does not prove the original clinical prototype is scalable.

The initial Supabase CLI inventory found the linked dental project inactive. The owner subsequently restored it through the dashboard. It has not been deleted or replaced. The public demo deliberately does not depend on that service. A free service that can pause on inactivity cannot be assumed to deliver a sustained availability commitment. See Supabase project pausing.

Workload definitions

“Simultaneous users” is ambiguous. Open tabs, actively browsing readers, authenticated sessions, concurrent HTTP requests and database transactions have different resource requirements. Agree on the workload before capacity acceptance.

The following is planning arithmetic: each active user performs one action every 30 seconds, with three HTTP requests per action. At a 95% cache hit rate, five percent of those requests reach the origin. A clinical API cannot apply that public-cache assumption to personalized responses.

- Active readers: 10,000 • Edge requests/second: 1,000 • Origin requests/second at 95% hit: 50
- Active readers: 100,000 • Edge requests/second: 10,000 • Origin requests/second at 95% hit: 500
- Active readers: 1,000,000 • Edge requests/second: 100,000 • Origin requests/second at 95% hit: 5,000

node scripts/capacity.mjs reproduces the table from config/tooling.json. The numbers are a workload model, not tested capacity. At zero cache hits, the million-reader scenario reaches 100,000 origin requests/second. If each response is 100 KB, that is approximately 10 GB/second before protocol overhead. Cold caches, launches and cache purges are meaningful failure scenarios.

Cloudflare does not cache HTML or JSON by default; eligible assets and origin directives must be checked separately. An origin public header alone does not prove edge HTML caching. Personalized records, authentication, exports and error responses must not enter a shared public cache. Verify cache behavior with response headers and separate cold/warm measurements. Cloudflare cache behavior.

Staged architecture

Stage A: current demonstration

Use a CDN for eligible immutable assets and a bounded static origin. Keep hashed asset filenames, compression, explicit routes, a small health endpoint, no public write API and no database dependency. Limit the origin container’s CPU, memory and process count and bind it to loopback behind HTTPS. Retain versioned releases and an off-server archive.

Admission gate: browser flows, read-only form boundary, headers, isolated deployment, health checks and restore/parity evidence. Run a small loopback HTTP test; do not stress a shared production server. Current measurements belong in VERIFICATION.md, not in this planning table.

Stage B: measured 10,000-reader target

Create an isolated test environment. Generate the agreed traffic mix including images, page transitions, cold requests and slow connections. Measure p50/p95/p99 latency, errors, origin CPU/memory, egress and cache hit rate. Repeat with a deployment in progress. Define pass thresholds before the test and retain generator saturation evidence; an overloaded load generator cannot establish server capacity.

If introducing real clinical workflows, split public delivery from the authenticated service immediately. Use a separately tested identity provider, staff MFA, short-lived sessions, server-side

role/patient/tenant scope, bounded payloads, timeouts, idempotency and audit persistence. Cache public service descriptions; never use those caches for patient responses.

Stage C: 100,000-reader and transactional growth

Introduce multiple origins across failure domains when measurements or availability requirements justify them. Regional ingress directs public requests to healthy origins. Clinical services remain stateless where practical; authoritative patient data stays in its approved region. Use a bounded connection pool, indexed tenant/subject queries, keyset pagination, query deadlines and per-operation concurrency limits.

Do not map one user to one database connection. PostgreSQL allocates resources based on connection limits, so indiscriminately increasing `max_connections` is not scaling. Benchmark the pool against actual query latency and database capacity. Replica lag must not make permissions, consent withdrawal or appointment confirmation stale. PostgreSQL connection settings.

Move notifications, large exports and noninteractive work into queues with idempotent consumers, retry budgets, backoff and a dead-letter path. A booking confirmation must be based on a committed authoritative record, not optimistic email delivery. Apply backpressure and return a controlled failure when a queue or downstream service is saturated.

Stage D: million-reader design target

At this scale, architecture and operations need separately funded capacity testing, regional delivery, traffic engineering and a staffed response model. Test hot keys, cold cache storms, failed regions, certificate/DNS incidents, database failover, queue backlog and restoration. Partition clinical data only after tenant boundaries, access patterns, regional law and recovery behavior are understood.

A million open static tabs is not a million simultaneous clinical transactions. The latter demands a different budget, workload model and data architecture. This repository offers a migration path and reproducible small-scale tools; it does not make either workload “easy” by assertion. Kubernetes is optional and is not installed merely to make the repository look sophisticated.

Availability objectives

A service-level indicator measures behavior; an objective defines its target; an SLA is a contractual commitment. Keep these separate. Google SRE: Service Level Objectives.

Proposed public availability objective: 99% of one-minute observation intervals over a rolling 30-day window have a valid TLS response and expected health body within the agreed latency limit. Count planned maintenance as unavailable unless a contract explicitly says otherwise. A 30-day window contains 43,200 minutes; one percent is 432 minutes, or 7 hours 12 minutes. Do not mix this time-based budget with a request-success denominator.

Clinical availability needs separate indicators for login, reading a permitted record, submitting an appointment and performing an authorized write. A static health page says nothing about database correctness. Measure from at least two independent locations and investigate disagreements; do not mask a regional outage by averaging it away.

The release includes `scripts/probe.mjs`, which emits a timestamp, success and latency without credentials or a patient identifier. Scheduling it on an independent always-on monitor is an operator task. A laptop that loses power is not an availability monitor. No 30-day observation period or external monitoring SLA is claimed by this deployment.

Recovery design

- Recovery concern: State • Public demo: Static release + configuration • Future clinical service: Database, uploads, keys, identity and audit state
- Recovery concern: Backup • Public demo: Off-server release archive with SHA256 manifest • Future clinical service: Application-consistent encrypted backups and protected key recovery
- Recovery concern: Restore test • Public demo: Extract archive and compare all release files • Future clinical service: Restore to isolated region-approved environment; test application reads/writes
- Recovery concern: RPO • Public demo: No user-generated clinical state in this release • Future clinical service: Proposed target set by client; validate against backup/replication cadence
- Recovery concern: RTO • Public demo: Measure actual release rollback/restore • Future clinical service: Proposed target set by clinical impact; include DNS, keys, identity and staffing
- Recovery concern: Failure domain • Public demo: One shared origin • Future clinical service: Multiple independently recoverable domains if funded

Do not advertise an unmeasured recovery time. Keep the previous release available. When restoring a clinical backup, reapply deletion/consent restrictions and legal holds where required; restoring a database must not silently resurrect permissions or erased data.

Cost and acceptance boundary

The demo reuses existing hosting and free tools. Existing hosting still has the owner's pre-existing cost; no new paid service is necessary for this static release. Free tiers do not promise unlimited capacity, contractual uptime or health-data processing approval. Before any paid upgrade, obtain explicit approval for price, scope and recurrence.

A production proposal should budget regional hosts, database capacity, backup storage, monitoring, security support, contractual vendor options, test infrastructure and operations. Provider price changes belong in configuration/procurement records, not hardcoded business logic. Progress from one stage only when measurements show why the next investment is necessary.

Deployment, monitoring and recovery

Build and inspect

Use Node 24 or newer within the supported toolchain. Run `npm ci`, `npm run check`, `npm audit`, `npm run test:browser` against a local preview, and `node scripts/security-check.mjs`. Run `gitleaks` over history and staged changes. Retain raw logs privately. `node scripts/build-deploy.mjs` generates Compose, nginx and gateway-site files from deployment settings and the canonical header policy.

The release consists only of `dist/`, `compose.yaml`, `nginx.conf`, `security-headers.conf` and the project gateway file. Do not upload source, environment files, credentials, memory, source maps, SQL or local research artifacts to the web root. Settings identify the hostname, pinned image, loopback port and resource bounds in one place.

Deploy safely

- Read the shared infrastructure inventory, then inspect current DNS, listeners, containers and capacity. Confirm project ownership and compare any existing project files with local snapshots.
- Build a new versioned release locally. Generate a SHA256 manifest, save an off-server archive and verify extraction against the manifest.
- Upload only the allowlisted release to its unique release directory. Validate Compose and nginx before starting the project container. Verify loopback health and inspect actual limits/bindings.
- Prepare only this project's gateway site locally. Preserve the shared main configuration and unrelated sites. Validate the whole gateway configuration before reloading.
- Create/update only the confirmed project DNS record. Verify direct-origin certificate validity using hostname SNI, then public TLS, expected content and security/cache headers.
- Compare every deployed release file hash with local. Record image digest, release, gateway hash and DNS change evidence privately.

Rollback

Retain the previous release and its manifests. Compare current project configuration first to avoid overwriting newer work. Start the previous project release with its own verified Compose file, then verify health, public content and hashes. Restore only this project's gateway/DNS values when they changed. Never remove unrelated containers or volumes, run a global prune or use down -v as a routine rollback.

This static release has no patient-generated persistent state. A later clinical release requires an application-consistent database and upload backup, keys, restored permissions/holds and compatible migrations; restoring static files does not recover patient data.

Monitoring

Run node scripts/probe.mjs <https://your-host/healthz> from an independently hosted scheduler and retain its JSON output. Configure a named responder and alert policy before making an operating SLA. The probe requires valid TLS and the expected body, returns nonzero on failure and does not print sensitive URLs. Do not count a one-time probe or server-local healthcheck as a month of availability.

The loopback load tool refuses remote targets. Increase load only in an explicitly isolated test environment after agreeing on resource limits. Never run a million-user test against this shared VPS or a free third-party project.

Recovery records

The private checkpoint stores deployment state before/after each significant action. The dossier stores durable incident lessons. CREDENTIALS.md is gitignored and must be reconciled privately when credentials change. Public verification contains counts and limits, not passwords, account IDs or server access details. Independent review is deferred to a separate session by the owner; do not record an invented approval.

Clinical prototype release gate

The supabase/ directory preserves historical clinical prototypes. Every Edge Function endpoint now returns a shared 503 boundary before processing a request. No environment variable can bypass it. The original function source was downloaded and all 15 recovered source files matched the Git baseline. The owner restored the historical project. After explicit owner approval, all 11 guards were deployed and each returned the expected HTTP 503 response. All 16 downloaded deployed source files matched local SHA256. No database records or migrations have been changed.

The following defects were recorded in the previous dossier and reconfirmed in inspected source. They are not falsely relabeled as repaired by disabling public access. Removing the release guard requires a dedicated clinical implementation, isolated test database and independent review.

- Blocker: Plaintext writers versus migration 010 • Required repair and verification: Reconcile every encrypted write/read with the final schema; empty and upgrade migration tests
- Blocker: Patient-selected query tables • Required repair and verification: Fixed operation/column allowlists, tenant/subject scope, row limits and negative authorization matrix
- Blocker: FHIR subject ownership • Required repair and verification: Enforce ownership for every patient-linked resource and search/export; test IDOR and care-assignment boundaries
- Blocker: Notification and breach authority • Required repair and verification: Verified caller/scheduler identity, fixed recipient/template policy, audited admin authority, no public service-role utility
- Blocker: Rate limiter • Required repair and verification: Atomic shared limiter, trusted proxy policy, fail closed on dependency errors and test concurrent requests
- Blocker: Consent • Required repair and verification: Purpose-specific grant/withdraw journey with legal-basis distinction, RLS and downstream enforcement
- Blocker: Payments • Required repair and verification: Server-owned catalog/amount, verified provider signatures, replay/idempotency handling; sandbox-only before separate approval
- Blocker: Retention • Required repair and verification: Jurisdiction-specific schedule, legal holds, dry run, approval, scheduler and restoration behavior
- Blocker: MFA/sessions • Required repair and verification: Actual staff MFA enforcement, revoked-role/session tests, token expiry and inactivity policy
- Blocker: Audit • Required repair and verification: Durable authorized server events, verified failure behavior, redaction, access review and tamper evidence
- Blocker: Configuration • Required repair and verification: Central typed server settings; inventory all Deno variables, limits, provider endpoints and templates
- Blocker: Operations • Required repair and verification: Vendor agreements, keys, encrypted off-server backups, restore rehearsal, monitoring and incident ownership

Do not apply historical migrations to an unknown live database. Inventory and export the schema/functions first, compare them with local source, preserve existing records, and build a tested upgrade/rollback plan. Never place service-role credentials in frontend configuration. Never send real patient data to a free-tier provider without verified legal and contractual suitability.

The frontend sample is deliberately not a substitute for this gate. Browser-selected roles only illustrate policy decisions. The static demo proves neither database RLS nor backend encryption nor clinical FHIR interoperability. These limits should remain visible to clients and developers.

Release verification and limitations

Evidence scope

This is a synthetic portfolio release, not a clinical production approval. Verification was performed in September 2026. Independent fresh-context review is deferred to a separate session at the owner's request.

- Gate: Automated regression tests • Result: 15/15 pass: access decisions, consent, hold/erasure, sample export, runtime boundary and source regressions
- Gate: Type check • Result: Pass; strict scope covers new runtime/policy/guard modules, not all historical JSX or Deno functions
- Gate: Lint • Result: Pass, zero errors/warnings in configured application/tooling scope
- Gate: Build • Result: Pass with locked Node/Vite toolchain; removed obsolete eval-producing animation dependency
- Gate: Dependency audit • Result: Zero reported npm vulnerabilities in the final lockfile audit; point-in-time evidence
- Gate: Gitleaks • Result: Zero findings in baseline history and scanned working source; final staged scan also passed with zero findings
- Gate: Security edit hook • Result: Exit 0 for application, tooling, automated tests and function sources
- Gate: Semgrep • Result: 23 security-audit rules, 106 scanned targets, zero findings; not a penetration test
- Gate: Bandit • Result: Not applicable to shipped code: no Python application source; private operator tooling is outside the shipped application
- Gate: Browser • Result: 24/24 local checks and 24/24 explicit-public-URL checks pass
- Gate: Accessibility • Result: Two axe checks on new main-content surfaces pass; mobile width/keyboard behavior exercised; full site conformance unverified
- Gate: Backend containment • Result: 11/11 deployed endpoints return HTTP 503 with expected guard response; all 16 downloaded source files match local SHA256
- Gate: Database preservation • Result: Only Edge Function code was deployed; no SQL, migration, record deletion or database mutation command was executed
- Gate: Independent review • Result: Deferred by owner; no approval verdict exists

The public browser suite checks 20 routes, no automatic third-party requests from application pages, disabled intake inputs, the synthetic patient workflow, sample export, denied operations, held erasure, reset and protected-route redirection. Cloudflare/network operational metadata remains outside the application's no-tracking claim.

Deployment and recovery

The origin runs a pinned nginx image as a non-root user with a read-only filesystem, dropped capabilities, no-new-privileges, loopback binding and explicit CPU/memory/process limits. DNS and gateway publication reused existing resources. No new paid plan, server or domain was purchased.

Before changes, legacy HTML and six entry assets matched the local baseline, and 15 recovered backend files matched Git. Versioned static archives are kept off-server and extraction is checked against SHA256 manifests. Each staged release has 138 allowlisted files, with local/remote hashes checked. Five unrelated gateway site files were preserved.

Public TLS initially failed while the new DNS record propagated; certificate issuance retried and public HTTPS recovered. HTTP checks found an nginx location-precedence error that prevented immutable caching on three entry assets. The redundant extension regex was removed locally and a new versioned release prepared. The corrected release passed all 138 file hashes. Direct-origin and public TLS were verified. Container recreation reached healthy status and the expected response in 37.15 seconds; this measures that command sequence, not a disaster-recovery guarantee. Actual runtime inspection verified user 101:101, read-only root, all capabilities dropped, no-new-privileges, 128 MiB RAM, 0.5 CPU and 64 processes. The container port was unreachable externally and five unrelated gateway files retained their hashes. Final public HTTP checks passed 21/21 conditions; see the retained release evidence.

Capacity evidence

A bounded localhost Vite HTTP microbenchmark used 20 clients and 400 requests: zero failures, about 1,735 requests/second, p95 21.51 ms over about 0.231 seconds. This tiny warm local sample does not establish production capacity, endurance, CDN behavior or clinical transaction throughput. It must not be extrapolated to 10,000 or one million simultaneous users.

The workload calculator models 10,000 / 100,000 / 1,000,000 active readers, giving 1,000 / 10,000 / 100,000 edge requests per second under its stated assumptions. These are calculations only. No large-scale production load test or 30-day availability measurement was performed. The 99% target permits 432 unavailable minutes per 30-day window; there is no contractual SLA.

Configuration and secret audit

Real secrets were present in private local configuration and recovery sources; they were kept outside the public artifact. No real secret was detected in scanned shipped source or Git history. The private credentials, dossier, environment files and memory directory are ignored. Credential recovery and any required rotations remain private operator responsibilities.

New runtime origin/mode and inactivity settings, sample workflow data, privacy disclosures, image mapping, deployment image/ports/limits and test/load assumptions have central owners. Provider clients cannot be enabled through environment flags in the public release. Protocol syntax, HTTP status codes and static routing syntax remain fixed intentionally.

This is not a complete refactor of every historical module: dormant clinical Deno environment reads, provider URLs, timeouts, encryption/role contracts and older presentation copy still require consolidation before clinical activation. The scoped type check does not prove these modules correct. See CLINICAL-RELEASE.md for the mandatory repair gates.

Acceptance accounting

Of the original 14 draft criteria, 10 are addressed by this synthetic release (1, 2, 6–11, 13, 14); document synchronization is recorded in the final handoff. Criteria 3–5 are only partially satisfied: the demo is isolated, but full clinical authorization/schema/configuration remediation remains blocked from production use. Criterion 12 is explicitly deferred to a separate review session. A

future reviewer must assess the final diff against these limitations; do not report 14/14 or production readiness.

Media licensing and provenance

The application serves 47 local stock-image variants. The repository media manifest preserves the exact source-to-file mapping. People shown are stock subjects, not verified practitioners or patients of the fictional clinic. See the Unsplash license.

- Stock photograph 1
- Stock photograph 2
- Stock photograph 3
- Stock photograph 4
- Stock photograph 5
- Stock photograph 6
- Stock photograph 7
- Stock photograph 8
- Stock photograph 9
- Stock photograph 10
- Stock photograph 11
- Stock photograph 12
- Stock photograph 13
- Stock photograph 14
- Stock photograph 15
- Stock photograph 16